> **Forms are NOT built by enabling feature flags.** Feature flags, form metadata, and field-level rules are three separate layers. Confusing them creates an unmaintainable system where every field change requires a deployment.

Three Layers of Configuration

1. Module Level

Feature Flags (LaunchDarkly) — Does this module exist for this tenant?

`program.nslp.enabled` **true** → Show NSLP nav item, load NSLP routes

`program.nslp.enabled` ~~**false**~~ → NSLP nav item hidden, routes 404, module code never loads

`platform.training.enabled` **true** → Training section appears in nav (HI only in DIS; all CTS states)

`claims.auto-decline.enabled` **true** → Auto-decline logic active in claim submission

`dircert.directver.enabled` **true (IN only)** → DirectVer sponsor-facing DC module shown; Indiana reference implementation

Granularity: module / feature / business rule. ***Not field-level.*** *Evaluated by Angular route guards and component-level LD directives.*

2. Form Level

Form Metadata (cnp_global FormDefinition) — What sections and fields exist?

`AnnualFormType: FSMC` → FSMC section only appears if sponsor.hasFsmcContract = true

`SiteApp section: NonCong` → Non-congregate section only if program = SFSP or (NSLP + PY ≥ 2024)

`ReviewVersion: 4.2 NSLP` → Question bank loaded from cnp_global ReviewVersions

`VCAScoringTool v3.2` → VCA checklist sections loaded from cnp_global VCAScoringToolItems

*Definitions in **cnp_global** (shared all states). Per-tenant overrides in **cnp_ops**. No deployment needed to change form structure.*

3. Field Level

Formly Field Config — Required / optional / hidden / conditional?

`enrollment` All states — must have enrollment for NSLP ***required***

`avgCostHalfPint` Required only if Special Milk Program active ***optional***

`nonCongregateDays` Visible only if nonCongMode ≠ 'congregate-only' ***conditional***

`stateMatchAmount` Hidden in states with no state match rate ***hidden***

*Resolved server-side per request: base schema from cnp_global + tenant override + program context. Returned as **FormlyFieldConfig[]** to Angular.*

Shell Initiation: Angular App Bootstrap

1 Subdomain Resolution

Browser hits mn.linqcnp.com. Route 53 → CloudFront → ALB. YARP extracts "mn" from Host header → resolves TenantId = "mn-doe".

```
Host: mn.linqcnp.com
→ TenantId: "mn-doe"
```

2 Keycloak OIDC

Angular checks for valid JWT in memory. If absent, redirects to ut-doe Keycloak realm → state IdP (AD/SAML/JWT) → returns JWT with

3 Tenant Config Load

Angular calls GET /api/tenant/config. API reads from cnp_global (387 shared reference tables). ICnpContext simultaneously resolves the state's cnp_ops Aurora connection (1,015

```
→ ICnpContext: GlobalDb (cnp_global) + StateDb (ut-doe.cnp_ops)
```

tenant_id, roles, programs claims.

```
jwt.tenant_id = "mn-doe"
jwt.programs = ["NSLP","CACFP"]
```

operational tables). Returns: agency name, logo S3 URL, brand color, LD bootstrap payload, cnp_ops connection confirmed.

```
{ agencyName: "Minnesota DOE",
  logoUrl: "s3://…/mn-doe/logo.svg",
  primaryColor: "#1BAAC1" }
```

4 Brand Tokens Applied

CSS custom properties set on :root. @linq/snackpaq components automatically pick up tenant brand. Logo element src updated. Agency name in nav header.

```
:root {
  --linq-primary: #1BAAC1;
  --linq-agency: "MN DOE"
}
```

5 LaunchDarkly Bootstrap

LD SDK initialized with tenant context {tenantId, stateCode, programs}. Bootstrap payload (from step 3) seeds all 47 flags instantly — no extra network call. (43 original + session replay ×3 + non-congregate SFSP)

```
ldClient.identify({
  kind:"multi",
  tenant:{key:"mn-doe"}
})
```

6 Nav + Routes Built

Angular route guards check LD flags. program.nslp.enabled → NSLP routes registered. platform.training.enabled = false → Training route removed. Nav items rendered conditionally.

```
canActivate: [FeatureFlagGuard]
flag: 'program.nslp.enabled'
```

7 Session Replay Active

LD Session Replay SDK starts recording. Privacy mode: mask-all by default. Student PII elements have ld-privacy-redact attribute.

Replay linked to LD tenant context.

```
privacySettings: {
  defaultMode: 'mask'
}
```

Form Rendering: Form Built at Runtime

1 User Navigates to Form

Program year is part of the URL path: **/2025-26/applications/nslp/sponsor-app**. Angular Router segments: `/:programYear/applications/:program/:formType`. This allows concurrent tabs across different program years without session conflict. tenantId is resolved via ICnpContext (subdomain), not in the URL.

2 API: GET /form-schema

Angular calls GET /api/forms/schema?formType=NslpSponsorApp&programYear=2026. Server resolves schema from cnp_global + tenant overrides.

```
GET /api/forms/schema
?formType=NslpSponsorApp
&programYear=2026
&tenantId=mn-doe
```

3 Server Builds Schema

1. Load base FormDefinition from cnp_global.
2. Apply tenant overrides (cnp_ops).
3. Apply program-year conditions (e.g., nonCong fields only if PY≥2024).
4. Return FormlyFieldConfig[].

4 Formly Renders

Angular Formly receives FormlyFieldConfig[]. Renders inputs, selects, date pickers, custom components. Required/hidden/conditional states applied from config — no if/else in template.

```
formly-form
 [fields]="schema"
 [model]="sponsorApp"
```

5 Live Conditional Logic

hideExpression and expressions evaluate against live form model. Changing "Does your sponsor use FSMC?" to Yes → FSMC section appears instantly without page reload.

```
hideExpression:
 "!model.usesFsmc"
```

6 Validation on Submit

Formly validates all visible required fields. Hidden fields are skipped automatically. Custom validators (enrollment cap, 60-day deadline) run via asyncValidators from API.

7 PDF Snapshot Triggered

On status change (Submitted/Approved), SQS event fires → Document Service renders PDF snapshot of the form as it appeared at this moment. Stored to S3.

```
event: ApplicationSubmitted
→ SQS → PDF render
→ S3 snapshot
```

PDF Strategy — User-Initiated Print + Automated Snapshots

User-Initiated Print (Forms)

Browser `window.print()` with **print CSS** (`@media print`). Navigation, sidebars, action buttons hidden via `display:none` . No server component. Covers: applications, budgets, management plans, claim summaries. Simplest — works in all browsers.

```
@media print {
 .nav, .sidebar, .action-bar { display:none; }
 .form-content { width:100%; }
}
```

Multi-Page Reports — IPdfRenderService (QuestPDF)

NA-44 federal reports, review reports, and audit exports use server-side headless Chromium via **QuestPDF** in the Document Service. API endpoint accepts a report token → renders → returns S3 pre-signed URL. Covers complex paginated documents where print CSS is insufficient.

```
POST /api/reports/render
{ reportType: "FNA-44", claimId: 123 }
→ SQS → DocumentService.IPdfRenderService
→ S3 pre-signed URL (15-min TTL)
```

Automated Status-Change Snapshots (existing)

On ApplicationSubmitted / ClaimLocked / ReviewClosed events, SQS triggers Document Service → IPdfRenderService → S3 immutable snapshot. Pre-signed URL returned on-demand. *This path already existed in the design — see flow step 7 above.*

> EvoPdf (legacy commercial library) is **not** carried forward. Print CSS covers the 90% case with zero infrastructure. QuestPDF covers the 10% of complex reports. The `IPdfRenderService` interface in the Platform Engine abstracts both — Document Service chooses the implementation at render time based on `ReportComplexity` metadata.

FormlyFieldConfig: Field-Level Required / Optional / Hidden

```
// Server returns this array for NslpSponsorApp — built from cnp_global + tenant overrides
const schema: FormlyFieldConfig[] = [
  {
    key: 'organizationName',
    type: 'input',
    props: {
      label: 'Organization Name',
      required: true,        // All states — always required
    }
  },
  {
    key: 'stateMatchAmount',
    type: 'currency',
    props: { label: 'State Match Amount', required: true },
    // Hidden for states with no state match (config-driven, not hardcoded)
    hide: false,              // true = UT (has state match), false = GA (no state match)
    expressions: {
      hide: '!config.stateMatchEnabled'   // resolved from tenant config
    }
  },
  {
    key: 'avgCostHalfPint',
    type: 'currency',
    props: {
      label: 'Avg Cost per Half Pint',
      required: false,       // Present but optional unless SMP enabled
      description: 'Required only if Special Milk Program is active'
    },
    expressions: {
      'props.required': 'model.programCodes?.includes("SMP")'
    }
  },
  {
    key: 'nonCongregateMealMode',
    type: 'select',
    props: {
      label: 'Non-Congregate Service Mode',
      required: false,
      options: [
        { label: 'Congregate Only', value: 'CONGREGATE' },
        { label: 'Non-Congregate Only', value: 'NON_CONGREGATE' },
        { label: 'Mixed', value: 'MIXED' }
      ]
    },
    // Hidden entirely for NSLP if programYear < 2024 (federal gate)
    expressions: {
      hide: 'model.programYear < 2024 || !model.programCodes?.includes("SSO")'
    }
  },
  {
    key: 'boardMemberCount',
```

```
    type: 'number',
    props: {
      label: 'Number of Board Members',
      required: true,
      min: 3,
      description: 'CACFP requires minimum 3 board members'
    },
    // Only required for CACFP sponsors — present but optional for SFSP/NSLP
    expressions: {
      'props.required': 'model.programCodes?.includes("CACFP")'
    }
  }
]
```

Field States

State	Formly Config	Behavior	Validation	Example in CNP
Required	`props.required: true`	Field shown with red asterisk (*). Form cannot submit if empty.	Formly adds RequiredValidator automatically. Custom validators also run.	Organization Name, Agreement Number, Enrollment count for NSLP sites
Optional	`props.required: false`	Field shown without asterisk. Can be left blank. Submits without error.	No required validation. Custom validators may still run (e.g., format check if value provided).	Average Cost per Half Pint (shown for all, required only if SMP active), Phone extension
Hidden	`hide: true` or `expressions.hide`	Field NOT in DOM. No label, no input. Cannot be tabbed to. Value reset to null if model changes to hide.	Skipped entirely — no required check, no format check. Safe for fields that don't apply.	State Match Amount (hidden in states with 0% match), Non-Congregate mode (hidden if PY < 2024 or not SSO)
Read-Only	`props.disabled: true` or	Field shown, value visible, but cannot be edited.	Skipped for user-facing validation. Server still	Agreement Number (system-generated, shown

	`props.readonly: true`	Appears grayed out.	validates on submit.	for reference), Approved ClaimLaws rate (frozen — display only)
Conditional	`expressions: { hide, 'props.required' }`	Field visibility or required state changes dynamically based on other field values or model state.	Required validation activates/deactivates in real time. Angular Formly re-evaluates on every model change.	FSMC section appears when usesFsmc = true; Non-cong distribution days appear when mode ≠ CONGREGATE; Board member count required when CACFP in programCodes

State-Specific Branding: Logo, Colors, Agency Name

How It Works: Tenant Config → CSS Custom Properties

Logo assets stored in S3 under `/{tenantId}/branding/logo.svg`. State agencies upload their own logo via a branding settings page. Served via CloudFront CDN with long cache-control headers. If no logo: LINQ default logo shown.

```
// 1. Tenant config loaded at bootstrap (GET /api/tenant/config)
interface TenantConfig {
  agencyName: string;          // "Minnesota Department of Education"
  logoUrl: string;             // S3 pre-signed URL or CDN URL
  primaryColor: string;        // "#1BAAC1" — Sky Blue for MN
  tenantCode: string;          // "MN"
}

```

Examples



```
// 2. Applied to :root in app.component.ts
applyBrandTokens(config: TenantConfig) {
  const root = document.documentElement;
  root.style.setProperty('--linq-primary',    config.primaryColor);
  root.style.setProperty('--linq-agency-name', config.agencyName);
  this.logoUrl = config.logoUrl;  // bound to <img> in nav
}

// 3. @linq/snackpaq components use CSS variables
// Buttons, nav highlights, active states all inherit --linq-primary
// No component code changes between states
```

Form Preview

The same `NslpSponsorApp` FormDefinition is loaded for all states. Utah sees State Match (because `config.stateMatchEnabled = true`). Georgia does not. Montana does not see the SSO non-congregate fields (because Montana does not participate in SSO). No code branches. Only metadata differences between tenants. Once again, this is only an example of what it COULD look like, not a design requirement.

